

장선규

Jang Sungyu · Frontend Engineer

사용자 지표와 기술 의사결정으로 설명 하는 프론트엔드 프로젝트

공개 가능한 개인 프로젝트를 중심으로, 프론트엔드 엔지니어로서 **성능·테크니컬 SEO·데이터 통신·인프라 운영** 문제를 어떻게 해결했는지 그 과정과 판단을 정리한 자료다. 각 프로젝트는 실제 사용자·개발자가 사용하는 운영 중인 서비스이며, 프론트엔드 구현부터 인프라·SEO·개발 프로세스 설계까지 직접 다뤘다. 회사 프로젝트의 상세 case study는 이력서와 sungyujang.com/work에 별도로 정리했다.

CONTENTS

01 Poke Korea — 실사용 트래픽·SEO·인프라

03 Devfolio v3 — 콘텐츠 중심 정적 사이트

02 ConfigDeck — 개발 프로세스 시스템화

04 Other Projects — 이전 사이드 프로젝트

xodm95@gmail.com · sungyujang.com · github.com/js93121

Poke Korea

Next.js 15 + GraphQL 기반의 한국어 포켓몬 백과사전. AWS 셀프 호스팅, 테크니컬 SEO, 이미지 최적화 파이프라인을 1인 개발로 구축·운영 중이다.

poke-korea.com · github.com/js93121/poke-korea · [Frontend](#) · [Backend\(GraphQL\)](#) · [Infra](#)

4.2만

MAU
(GA4, 2026.07 기준)

189만

3개월 누적 노출
(Search Console)

4.56만

3개월 누적 클릭
(Search Console)

6.6

평균 게재순위
(Search Console)

프로젝트 동기

이전에 만든 Pokemon Dogam을 운영하며 마주한 기술적 한계(특히 SEO-캐싱)와 UX 문제를 개선하기 위해 시작했다. 단순한 기능 재구현이 아니라, "실제 사용자가 꾸준히 찾아오는 서비스"를 만들고 운영하는 것을 목표로, SEO를 통한 자연 유입·AWS 인프라 구성·이미지 최적화까지 서비스 전체 라이프사이클을 직접 설계·구현했다.

기술 스택

Frontend	Next.js 15 (App Router), TypeScript, React, Tailwind CSS
Data Layer	Node.js, GraphQL, Apollo Client, graphql-codegen
Infra	AWS (EC2, CloudFront, Lambda@Edge), PM2
Data Source	PokeAPI (수집·가공 후 자체 DB·GraphQL 서버로 재구성)

시스템 구조



01 가상화 렌더링 → 커서 기반 무한 스크롤 전환

전작 Pokemon Dogam에서 386개 카드를 한꺼번에 렌더링하던 문제를 `react-virtuoso` 가상화로 해결해 **스크립팅**

2,000ms→310ms(85%↓), 초기 렌더링 **386→6개**로 줄였다. 다만 가상화는 화면 밖 항목을 DOM에서 제거하므로 검색 크롤러가 콘텐츠를 보지 못하고 캐싱에도 불리했다. 자연 유입이 목표인 Poke Korea에서는 가상화를 제거하고 커서 기반 무한 스크롤 + 캐싱 전략으로 전환해, 렌더링 성능·SEO·UX의 균형을 다시 잡았다.

02 타입 안전한 GraphQL 통신 + 자체 데이터 레이어

Apollo Client와 `graphql-codegen` 으로 타입 안전한 데이터 통신을 구성하고, 화면별로 필요한 필드만 정확히 요청하도록 설계. 외부 API의 네트워크 지연·호출 제한이 서비스 운영 리스크가 되지 않도록, PokeAPI 원본을 수집·가공해 **자체 GraphQL 서버·DB로 재구성**했다.

03 AWS 셀프 호스팅 + 엣지 이미지 최적화 파이프라인

EC2 + PM2로 셀프 호스팅 환경을 구성하고, `CloudFront` + `Lambda@Edge` 로 원본 이미지를 실제 노출 크기에 맞춰 리사이즈하고 WebP로 변환·전달. 불필요한 대역폭 소비를 줄여 로딩 속도를 개선했고, 이 과정에서 CDN 캐싱과 엣지 컴퓨팅이 대역폭·속도·비용에 미치는 효과를 직접 확인했다.

04 테크니컬 SEO + 데이터별 차등 캐싱

페이지별 고유 메타 태그와 상세 페이지 동적 OG 이미지 서버 생성, JSON-LD 구조화 데이터, sitemap-robots 자동화를 적용. 캐싱은 데이터 성격에 따라 나눴다 — **포켓몬 데이터는 버전 갱신 전엔 불변이라 1년 장기 캐싱**, 매일 갱신되는 홈은 1일 + `stale-while-revalidate` 로 분리.

운영 성과

실제 사용자 트래픽 기반으로 성장을 추적 중이다. Google-네이버 두 검색엔진에서 안정적인 자연 유입을 확보하고 있으며, 콘텐츠 추가와 SEO 개선에 따라 **MAU가 2026년 6~7월 사이 2.5만 → 4.2만으로 증가**했다. (Search Console 3개월: 노출 189만·클릭 4.56만·평균 순위 6.6)

회고

기술 선택의 트레이드오프

같은 가상화 기술을 전작에선 도입하고 Poke Korea에선 제거했다. 렌더링 성능만 기준으로 보면 적절한 선택이더라도, SEO와 캐싱까지 고려하면 다른 결론이 나올 수 있음을 도입과 제거를 모두 해보며 확인했다.

1인 운영으로 얻은 것

인프라 구성·SEO 전략·트래픽 분석까지 서비스 전체 라이프사이클을 직접 다루며, 지표를 근거로 다음 개선을 결정하는 사이클을 경험했다.

남은 과제

평균 게재순위 6.6으로 안정적인 자연 유입은 확보했으나, 상위 노출을 더 끌어올리려면 콘텐츠 확장과 내부 링크 구조 개선이 남아 있다. 현재 성장세는 완성된 결과가 아니라 진행 중인 지표다.

ConfigDeck

ESLint·Prettier·TypeScript 등 설정 파일을 UI에서 골라 다운로드하는 웹 서비스. 서비스 자체는 단순하지만, **1인 개발에서 부족해지기 쉬운 기획·QA·코드 리뷰·의사결정 기록을 하네스와 문서화 체계로 보완**한 실험이다. 여기서 하네스는 프로젝트 규칙·역할·워크플로우·검증 기준을 문서와 자동화로 묶은 개발 보조 시스템을 뜻한다.

configdeck.dev · github.com/jsg3121/configDeck · [하네스 설계](#) · [기획](#) · [개발](#) · [QA](#) · [배포](#)

핵심 실험

1인 개발에는 코드 리뷰·QA·의사결정 논의·전문 분업이 구조적으로 부재하다. 이 공백을 **개발 프로세스로 시스템화**하면 어디까지 메울 수 있을까 — 이 가설을 검증하기 위해 빈 폴더에서 시작해 **2주 만에 배포**까지 완료했다(퇴근 후·주말 활용). 서비스 기능보다는, 1인 개발에서 팀의 프로세스를 어떻게 재현할지에 초점을 맞춘 프로젝트다.

기술 스택

Framework	Astro 6, Svelte 5 (Interactive UI)
Styling·Lang	TailwindCSS 4, TypeScript
Testing	Vitest, Playwright
Deploy·Tooling	Cloudflare Pages, Claude Code Harness

하네스 시스템 — 개발 프로세스 설계

AI에 작업을 위임한 게 아니라, `.claude/` 폴더에 **74개 하네스 문서로 협업 구조·역할 분담·의사결정 기록을 계층적으로 시스템화**했다. 각 폴더의 `index.md`를 먼저 읽도록 설계해, 사람이 새 코드베이스에서 README부터 읽는 패턴을 그대로 옮겼다.

```
.claude/
├─ ia/           기획 · 정보구조
├─ decisions/    ADR 13개 — 결정의 맥락 보존
├─ conventions/  코딩 · 스타일링 · 워크플로우 규칙
├─ seo/          SEO 가이드
├─ skills/       9 스킬 — 반복 워크플로우 슬래시 커맨드
├─ agents/       12 에이전트 — 부재한 동료 역할 (4 카테고리)
├─ qa/           QA 하네스
└─ hooks/        자동화
```

12 에이전트 · 협업 파이프라인

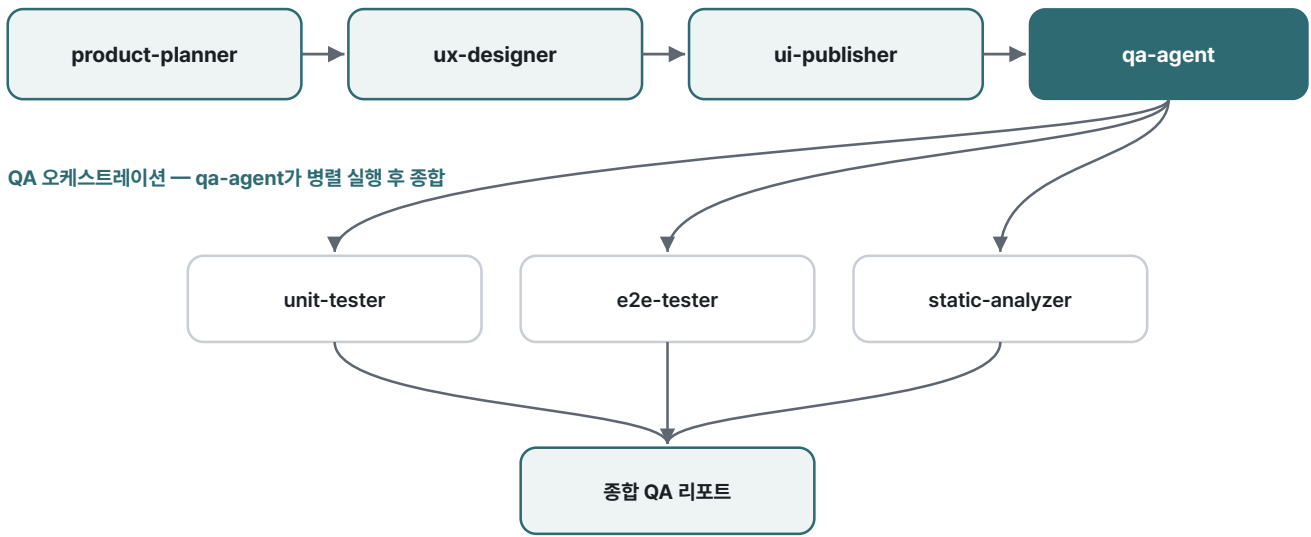
기획 (1) product-planner — 기능 SPEC 작성

개발 (4) config-maker · ui-publisher · ux-designer · seo-specialist

QA (4) qa-agent(총괄) · unit-tester · e2e-tester · static-analyzer

비즈니스 (3) market-intelligence · business-analyst · strategy-planner

순차 파이프라인 — 새 기능 개발



부재한 동료 역할을 4개 카테고리 에이전트로 정의하고, 순차 파이프라인·병렬 Fan-out/Fan-in-QA 오케스트레이션 3개 협업 패턴으로 처리하도록 설계

설계에서 특히 신경 쓴 원칙

· Why-First — 옛지 케이스에서도 판단 가능하게

모든 하네스 문서에 "왜 그런지"를 먼저 명시했다. 규칙만 나열하면 기계적으로 따르지만, 이유를 함께 주면 명시되지 않은 옛지 케이스에서도 올바르게 판단한다. 예: "컴포넌트 파일명은 PascalCase → Astro/Svelte 컴포넌트와 일반 유틸을 파일명만으로 구분하기 위함". 사람 동료에게 컨벤션을 전수하는 방식과 같다.

· 13 ADR — 결정의 맥락을 코드 외부에 보존

프레임워크(Astro+Svelte)·패키지 매니저(pnpm)·배포(Cloudflare Pages)·옵션 스키마 재설계 등 주요 결정을 ADR로 기록. 1인 개발에서 가장 잃기 쉬운 "왜 이렇게 했지?"의 맥락을 미래의 나와 에이전트가 함께 참조하도록 설계했다.

효과가 있었던 부분

01 UX 리서치 → 디자인 의사결정의 명확화

디자이너 없이 혼자 개발하면 "이게 좋은 디자인인가?"의 확신이 부족하다. 리서치 단계에서 경쟁 서비스를 분석하고 HTML 목업으로 선택지를 만들어 비교하니, 디자인 의사결정의 근거가 명확해졌다.

02 병렬 QA로 옛지 케이스 사전 차단

1인 개발에서는 검증 범위가 축소되기 쉬운데, 단위·E2E·정적 분석을 병렬로 돌리는 QA 오케스트레이션으로 놓치기 쉬운 케이스를 잡았다. 예: "빈 옵션 선택 시 다운로드 버튼이 활성화되는" 버그를 배포 전 발견·수정.

03 PR 생성 단계의 크로스체크

`/create-pr` 워크플로우에 "검증을 진행했는지" 확인 단계를 넣었다. 검증을 생략하기 쉬운 1인 개발의 한계를 프로세스로 보완해, 검증 없이 올라가는 PR을 줄였다.

회고 — 기대와 달랐던 것

컨벤션을 명시하지 않으면 코드 스타일이 분산된다

명시적인 컨벤션 없이는 생성 결과의 코드 스타일 일관성이 유지되지 않았다. 비슷한 로직도 매번 다른 스타일(어떤 파일은 `function` 선언, 어떤 파일은 화살표 함수)로 작성됐다. 세부 컨벤션을 처음부터 정의하는 것이 시스템 품질을 좌우한다는 점을 확인했다.

역할을 정의해도 의도 불일치가 남는다

역할을 명확히 정의해도 가끔 의도와 다르게 동작했다 — 간단한 검사 요청에 전체 분석을 시작하거나, 복잡한 요청에 표면적 답변만 돌아오는 경우.

description 설계의 정밀도와 역할 구분이 시스템 품질을 좌우한다는 점을 확인했다.

2주 실험의 결론

하네스는 AI를 프로젝트에 맞게 튜닝하는 시스템이다 — 맥락(CLAUDE.md)·전문성(에이전트)·자동화(스킵)·일관성(컨벤션)·의사결정 기록(ADR). 완벽하진 않지만, 1인 개발의 품질 검증과 의사결정 보완에 유의미했다는 것이 2주 실험의 결론이다.

Devfolio v3

경력과 기술 아티클을 담은 콘텐츠 중심 정적 사이트. sungyujiang.com을 구동하며, 타입 안전한 콘텐츠 구조와 정적 빌드·테크니컬 SEO에 초점을 맞춰 세 번째 버전으로 재구축했다.

[sungyujiang.com](#) · [github.com/js93121/devfolio_v3](#) · [콘텐츠 아키텍처](#) · [정적 빌드](#) · [SEO](#)

동기 · 기술 스택

이전 버전의 콘텐츠 확장성·SEO·유지보수 한계를 개선하기 위해 재구축했다. 글이 늘어도 구조가 무너지지 않고, 검색 유입이 잘 되며, 새 글 추가가 파 일 하나로 끝나는 구조를 목표로 삼았다.

Framework Astro, MDX, TypeScript, Tailwind CSS

Content Astro Content Collections (스키마 검증 기반 타입 안전 콘텐츠)

Focus 정적 사전 렌더링 · 테크니컬 SEO · 콘텐츠 파이프라인

ENGINEERING HIGHLIGHTS

01 타입 안전한 콘텐츠 구조

Content Collections 스키마로 글의 프론트매터(제목·날짜·태그·시리즈 등)를 빌드 타임에 검증. 잘못된 메타데이터가 있으면 배포가 아니라 빌드에서 걸러져, 콘텐츠가 늘어도 데이터 일관성이 유지된다.

02 MDX 아티클 파이프라인

글 안에 실제 코드·표· 다이어그램을 직접 임베드할 수 있는 MDX 기반 구조. "동작 원리를 글로 설명하는"(→ 04 Tech Writing) 글쓰기를, 텍스트에 그치지 않고 코드와 함께 보여주는 기반이 된다.

03 정적 빌드 + 테크니컬 SEO

전 페이지를 사전 렌더링해 빠른 LCP를 확보하고, 페이지별 메타·OG·JSON-LD·sitemap을 자동 생성. 회사에서 다룬 테크니컬 SEO 역량을 개인 사이트에도 동일하게 적용해, 기술 글이 검색에 노출되도록 설계했다.

CONFIGDECK과의 차이

ConfigDeck이 하나의 하네스 시스템을 설계한 실험이라면, Devfolio v3는 단계별로 다른 AI 도구를 조합한 워크플로우 실험이다 — 기획·글 구조·구현·교정 단계에 각기 다른 도구를 배치해보고 어떤 조합이 효율적인지 검증했다. 도구는 수단이고, 콘텐츠 구조와 정보 설계 자체는 직접 잡았다.

회고

콘텐츠가 늘수록 구조 설계(태그·시리즈·글 간 연결)가 기능 구현보다 중요해진다는 점을 확인했다. 다만 아직 축적된 글이 많지 않아, 이 구조의 효과는 콘텐츠가 더 축적되면서 검증될 예정이다.

Other Projects

이전에 진행한 사이드 프로젝트들. 각 시점의 기술 관심사와 시도를 정리했다.

상세 case study는 sungyujiang.com/side-projects

프로젝트 아카이브

Pokemon Dogam

2022.12 – 2023.04 · React + TS + GraphQL

React·TypeScript·GraphQL·MobX 기반 풀스택 프로젝트. 386개 카드를 한꺼번에 렌더링하던 문제를 `react-virtuoso` 가상화로 해결해 **스크립팅 2,000ms→310ms(85%↓)**를 달성했다. 다만 운영하며 가상화가 검색 크롤링·캐싱에 불리하다는 한계를 확인했고, Poke Korea에서는 이를 제거하고 커서 기반 무한 스크롤로 전환했다.

Devfolio v2

2022.06 – 2022.08 · Next.js + TS + SCSS

Next.js 기반 SEO 최적화 포트폴리오. GSAP 스크롤 애니메이션, WebP 이미지 최적화, lazy loading, 시맨틱 마크업을 적용해 **당시 Lighthouse 데스크탑 기준 전 항목 100점**을 달성. 이때 익힌 SSR·SEO 최적화 경험이 이후 실무의 테크니컬 SEO 작업으로 이어졌다.

Best Weather

2021.07 – 2021.12 · Vue 3 + TS + Docker + Prisma

Vue 3 Composition API·Docker·Prisma·MySQL 기반 위치 기반 기상 정보 비교 서비스. 브라우저 위치 정보로 현재 위치를 탐지하고, 기상청·외부 API를 비교해 시간별·주간 예보를 제공하는 풀스택 프로젝트.

Seoul Culture

2020.08 – 2020.10 · Vue + JS + 공공데이터 API

공공데이터 API를 활용한 서울 문화공연 정보 검색·탐색 서비스. 비동기 데이터 처리, 카테고리·지역·날짜 검색, 슬라이드·무한 스크롤을 라이브러리 없이 직접 구현.

Devfolio v1

2020.08 – 2020.10 · Vue + JS + SCSS

첫 개인 포트폴리오. 인트로 모션·프로젝트 슬라이드 애니메이션 등 시각적 인터랙션을 최소한의 라이브러리로 직접 구현. Devfolio 시리즈(v1 인터랙션 → v2 SSR·SEO → v3 콘텐츠 구조)의 시작점.
